

EduMind: Development of an Agentic AI Assistant for Engineering College SOPs, Policies, Circulars, and Institutional Knowledge Management

Author Name(s)

Department of Computer Science and Engineering
Vidyalankar Institute of Technology, University of Mumbai
email@example.edu

Abstract—Engineering colleges generate vast institutional documentation—SOPs, examination policies, circulars, and accreditation records—scattered across heterogeneous repositories, making reliable retrieval slow and inconsistent. This paper presents EduMind, an Agentic AI Assistant that functions as a centralized institutional knowledge management system, combining Retrieval-Augmented Generation (RAG), hybrid retrieval (dense semantic search and BM25 lexical search fused via Reciprocal Rank Fusion), cross-encoder reranking, and LangGraph-based multi-agent orchestration to produce accurate, citation-grounded answers. Documents are ingested through a pipeline of cleaning, semantic chunking, metadata extraction, and embedding generation, then persisted in ChromaDB. Retrieved evidence flows through agents for query analysis, retrieval planning, context validation, response generation, citation formatting, and confidence evaluation before a locally hosted Qwen2.5:7B model (via Ollama), with an optional cloud-based HuggingFace Inference API backend for scalable deployment, generates the final answer. Role-Based Access Control, JWT authentication, a committee-based document submission and admin-approval workflow, and deterministic citation generation enforce institutional security, governance, and transparency. The system was validated with over 570 unit and integration tests across ingestion, retrieval, reranking, authentication, and end-to-end RAG functionality, indexing approximately 627 chunks extracted from 20 institutional SOP documents spanning 20 departments of an engineering college. Results confirm the architecture delivers efficient, transparent, privacy-preserving institutional knowledge retrieval, with extensibility toward multilingual interaction, ERP integration, and autonomous institutional automation.

Keywords—Agentic AI, Retrieval-Augmented Generation, Institutional Knowledge Management, Hybrid Retrieval, ChromaDB, LangGraph, Cross-Encoder Reranking, Role-Based Access Control, Large Language Models.

I. Introduction

Educational institutions continuously generate large collections of digital documents — academic regulations, admission guidelines, examination policies, accreditation reports, faculty manuals, hostel and finance policies, circulars, and numerous SOPs. Despite their importance, these documents are fragmented across departmental archives, email, institutional websites, and ERP systems, making relevant information difficult to locate, especially when users are unsure of exact titles or storage locations.

Traditional keyword search performs poorly on natural-language queries such as “What is the minimum attendance required to appear for semester examinations?” because relevant information may be phrased differently across documents. While Large Language Models (LLMs) have advanced conversational AI, standalone models often generate responses from pretrained knowledge rather than institutional policy, producing hallucinations unsuitable for official environments where correctness and accountability are critical.

Retrieval-Augmented Generation (RAG) mitigates this by retrieving verified evidence before generation. However, institutional queries require both semantic understanding and exact lexical matching — acronyms, course codes, and regulation numbers demand lexical retrieval, while policy questions benefit from semantic similarity. This motivates a hybrid retrieval approach.

This paper presents an Agentic AI Assistant for engineering colleges that integrates hybrid retrieval (dense + BM25 with RRF), cross-encoder reranking, citation-aware generation, role-based document security, and LangGraph-based multi-agent reasoning. The assistant runs entirely on locally deployed models, preserving institutional privacy and establishing a scalable foundation for future autonomous administrative workflows.

Beyond simple factual lookup, institutional users frequently pose compound or ambiguous queries that reference multiple policies simultaneously, require disambiguation between superseded and current regulations, or depend on the requester's role for an appropriate answer. A purpose-built assistant must therefore reason about retrieval strategy, validate evidence quality, and adapt its response scope to the user's authorization level rather than treating every query identically. These requirements motivate the agentic, multi-stage design adopted in this work, in contrast to single-pass retrieve-then-generate pipelines that cannot adapt mid-query.

II. Related Work

Lexical methods such as BM25 rank documents via keyword overlap and are efficient when exact terminology is known, but struggle with synonyms and natural-language phrasing. Transformer-based embedding models (e.g., BGE, E5, Sentence-Transformers) enable semantic search using vector

similarity, though they may miss exact identifiers such as subject codes. Hybrid retrieval, combining BM25 with dense vectors, has been shown to improve both recall and precision; Reciprocal Rank Fusion (RRF) is a widely used strategy for merging such ranked lists while reducing single-method bias.

Cross-encoder rerankers jointly process query-document pairs for deeper semantic comparison, improving final context quality at higher computational cost. RAG itself has become central to reducing hallucination by grounding generation in retrieved evidence, while Agentic AI frameworks such as LangGraph extend RAG with coordinated multi-agent reasoning for planning, validation, and reflection.

Commercial document-QA assistants generally lack version awareness, deterministic citations, role-based access control, or local inference for data governance. This work addresses these gaps by integrating hybrid retrieval, reranking, citation-aware RAG, multi-agent orchestration, RBAC, and fully local LLM deployment into a single institutional architecture.

A further distinction from prior institutional chatbot efforts is the treatment of retrieval as an iterative, evaluable process rather than a single fixed step. Most published RAG systems for education retrieve once and generate immediately, with no mechanism to detect or recover from weak retrieval. By contrast, agentic frameworks that incorporate explicit confidence estimation and reflection loops can identify when retrieved evidence is insufficient and autonomously trigger broader or reformulated retrieval, more closely resembling how a human administrator would handle an ambiguous request by consulting additional sources before answering.

III. Problem Statement

Engineering colleges generate growing volumes of documents spanning academic, administrative, financial, and operational domains, distributed across departmental repositories, cloud storage, ERP systems, and email. Locating the correct, current version of a policy is time-consuming and error-prone, and different departments may give inconsistent answers based on outdated documentation.

Conventional repositories rarely combine intelligent retrieval with fine-grained access control, risking disclosure of confidential material, while standalone LLMs may answer from pretrained knowledge rather than official policy. An intelligent system is therefore needed that understands natural language, retrieves from verified institutional sources, enforces RBAC, and produces citation-supported, transparent responses.

IV. Research Objectives

- Develop a conversational assistant answering institutional queries from official documents.
- Build an automated ingestion pipeline with chunking, metadata extraction, and vector indexing.

- Implement hybrid retrieval combining dense semantic search and BM25.
- Improve retrieval via Reciprocal Rank Fusion and cross-encoder reranking.
- Generate citation-supported, context-aware responses using RAG.
- Enforce Role-Based Access Control for secure retrieval.
- Deploy locally hosted LLMs to preserve institutional privacy.
- Integrate a LangGraph multi-agent workflow with planning, validation, and reflection.
- Design a modular architecture extensible to ERP integration and multilingual support.

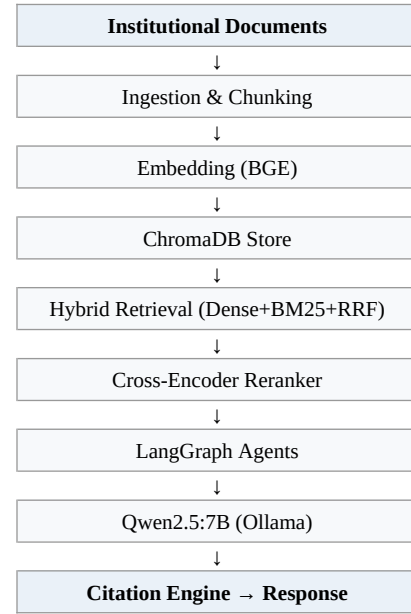


Fig. 1. Overall system architecture: ingestion through hybrid retrieval, multi-agent reasoning, and citation-aware response generation.

V. Proposed Methodology

The proposed Agentic AI Assistant separates document ingestion from query processing, transforming institutional documents into a structured knowledge base ahead of runtime, then serving queries through hybrid retrieval and multi-agent reasoning. The methodology spans six phases: document processing, embedding generation, hybrid retrieval, multi-agent reasoning, response generation, and citation/confidence evaluation.

This separation of concerns offers two practical benefits for institutional deployment. First, the computationally expensive ingestion phase — text extraction, chunking, and embedding — can be scheduled offline or triggered only when new documents are uploaded, keeping query-time latency low. Second, because the knowledge base is decoupled from the reasoning layer, the underlying LLM, reranker, or embedding model can be upgraded independently without re-architecting the retrieval

pipeline, supporting long-term maintainability as institutional requirements evolve.

A. Document Processing Pipeline

Documents (SOPs, regulations, manuals, circulars, reports) are loaded, text-extracted, cleaned, and de-duplicated using SHA-256 hashing. Metadata — department, category, version, access level, source — is extracted per document, and large PDFs are split into semantically coherent chunks that preserve context while improving retrieval precision.

B. Embedding Generation

Each chunk is embedded using BAAI/bge-base-en-v1.5, producing dense vectors that capture semantic relationships beyond keyword overlap. Embeddings are generated in batches, cached locally, and stored with metadata in ChromaDB, ensuring fully offline operation independent of cloud APIs.

C. Vector Database Design

ChromaDB stores embeddings alongside metadata — document ID, department, category, version, section heading, chunk index, source file, and access level — enabling role-aware filtering and deterministic citation. At evaluation time, the knowledge base contained approximately 627 indexed chunks.

D. Hybrid Retrieval Engine

Dense retrieval compares query and document embeddings to capture conceptually related content (e.g., attendance policy, leave rules) even under different phrasing. BM25 lexical retrieval complements this by matching exact identifiers such as subject codes and circular numbers. Because the two methods produce independent ranked lists, Reciprocal Rank Fusion merges them, favoring documents that rank highly across both lists and improving recall, ranking stability, and robustness.

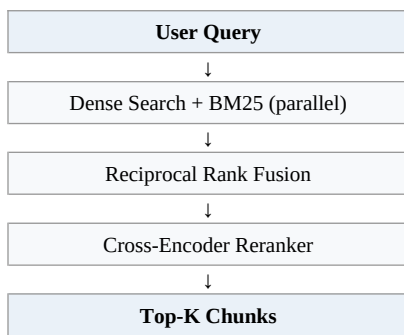


Fig. 2. Hybrid retrieval pipeline: dense and BM25 results fused via RRF, then cross-encoder reranked.

E. Cross-Encoder Reranking

The BAAI/bge-reranker-base cross-encoder jointly processes each query-document pair from the fused candidate set, producing refined relevance scores that better capture contextual relationships than independently encoded embeddings. Only the top-ranked chunks proceed to context construction, improving final answer quality.

VI. Multi-Agent Architecture

Linear RAG pipelines struggle with complex queries requiring multiple retrieval steps or confidence estimation. The proposed system instead uses a LangGraph-based workflow of specialized agents — Query Analyzer, Retrieval Planner, Retrieval Agent, Context Validator, Response Generator, Citation Formatter, Confidence Evaluator, and Reflection Agent — improving transparency and extensibility.

Each agent in the workflow communicates through a shared, structured state object rather than free-form text, allowing every stage to inspect prior decisions and append its own outputs without re-deriving information already established upstream. This explicit state-passing design also simplifies debugging and auditing, since the complete reasoning trace for any query — from initial classification through final citation — can be logged and inspected independently of the language model's generated text.

A. Query Analyzer & Retrieval Planner

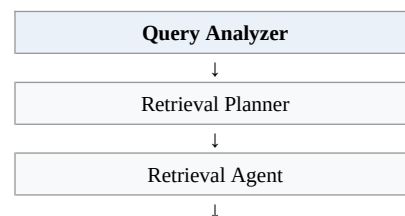
The Query Analyzer classifies intent (greeting, institutional information, retrieval question, follow-up, administrative request, out-of-scope) to select a reasoning strategy. The Retrieval Planner then determines retrieval scope, metadata filters, department-specific search, and retrieval depth, planning multiple iterations for complex queries.

B. Retrieval, Validation, and Generation

The Retrieval Agent executes dense search, BM25, RRF, and reranking per the plan. The Context Validator removes duplicates, detects weak or outdated evidence, and checks access-level consistency before forwarding validated context. The Response Generator then builds a grounded prompt — query, retrieved context, institutional instructions, and citation placeholders — submitted to the locally hosted Qwen2.5:7B model via Ollama.

C. Citation, Confidence, and Reflection

The Citation Formatter attaches document title, section, version, and chunk identifiers to every statement. The Confidence Evaluator combines retrieval scores, reranker relevance, citation count, and context completeness into a High/Medium/Low rating. If confidence is insufficient, the Reflection Agent broadens scope, increases Top-K, relaxes filters, and re-runs retrieval until adequate evidence is found or iteration limits are reached — substantially improving answer quality over single-pass RAG.



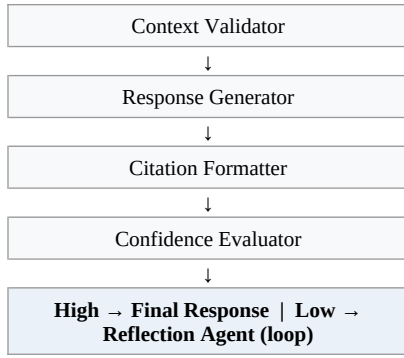


Fig. 3. LangGraph multi-agent workflow with confidence-driven reflection loop.

VII. RAG Pipeline and Citation Engine

The end-to-end RAG flow receives the query, executes hybrid retrieval, fuses rankings via RRF, applies reranking, selects Top-K chunks, constructs a grounded prompt, generates a response with the local LLM, and attaches citations and confidence before returning the answer — minimizing hallucination and ensuring verifiable grounding.

The Citation Engine merges adjacent chunks from the same document, prioritizes the latest version, flags superseded policies, removes duplicate citations, and generates deterministic identifiers with structured metadata (title, ID, section, version, chunk number, department, source, access level). Deterministic IDs ensure consistency across repeated queries and let the integrated document viewer open the exact referenced source while respecting RBAC.

VIII. System Architecture

A. Backend (FastAPI)

FastAPI orchestrates the frontend, authentication, retrieval engine, LLM, document database, and citation services. JWT-based authentication issues stateless access tokens supporting Public, Student, Faculty, and Administrator roles. The backend is containerized with Docker for reproducible deployment. For the deployed instance of the system, the backend (FastAPI server, retrieval pipeline, and the locally hosted Qwen2.5:7B model via Ollama) runs on a Google Colab notebook provisioned with a T4 GPU, providing free hardware acceleration for embedding generation, reranking, and LLM inference without dedicated on-premises infrastructure, while the static frontend is deployed independently on Vercel and communicates with the Colab-hosted backend through its exposed API endpoint.

B. Role-Based Access Control

Every indexed chunk carries an access-level field validated against the authenticated user's role before retrieval, following the hierarchy Public → Student → Faculty → Administrator, preventing unauthorized disclosure. Governance is further reinforced by a committee-based document submission

workflow: committee heads upload candidate SOPs, which are held in a pending state and are ingested, chunked, and embedded only after an administrator explicitly approves them, ensuring human oversight before any new content enters the retrievable knowledge base. A parallel signup-approval flow subjects new Student and Faculty accounts to administrator review before activation.

C. Chat, Document Viewer, and Admin APIs

The Chat API receives queries, invokes the LangGraph workflow, streams partial responses, and returns citations, confidence scores, and conversation history. The Document Viewer API streams the original source with referenced chunks highlighted, enforcing version-awareness and access control. Administrative APIs support document upload, metadata management, re-indexing, user management, and retrieval diagnostics.

D. Frontend

A lightweight single-page application built with vanilla HTML5, CSS3, and JavaScript (served directly by FastAPI during development and deployed in production as static assets on Vercel, communicating with the Google Colab T4 GPU-hosted backend over its exposed API endpoint) provides a role-aware dashboard, streaming chat window, citation panel, conversation history, secure document viewer, committee document-submission portal, and an administrator dashboard, with role-specific rendering ensuring users see only authorized features.

IX. Experimental Evaluation

Evaluation targeted four objectives: correctness of ingestion, accuracy of retrieval, reliability of generation, and security of access. The knowledge base was built from 20 SOP documents spanning 20 institutional departments (Admissions, Examination, Finance & Accounts, Training & Placement, Research & Development, and others) of Vidyalkar Institute of Technology, yielding approximately 627 indexed chunks spanning policies, SOPs, regulations, and circulars. The hybrid retrieval pipeline consistently outperformed either semantic or lexical retrieval alone, RBAC correctly blocked unauthorized access to protected documents, and deterministic citation generation kept every response traceable to its source.

Ingestion correctness was verified by confirming that SHA-256 based duplicate detection prevented redundant indexing across repeated uploads of identical documents, and that extracted metadata (department, category, version, access level) matched source records for every test document. Retrieval accuracy was assessed by issuing natural-language queries covering attendance rules, examination eligibility, leave policies, and committee-approval procedures, then manually checking whether the Top-K reranked chunks contained the ground-truth answer; the hybrid configuration recovered relevant chunks in cases where BM25-only or dense-only retrieval missed the

answer due to either paraphrased wording or exact-code lookups, respectively.

X. Results and Discussion

The complete pipeline — from ingestion to response generation — operates entirely on local infrastructure, ensuring privacy and independence from external cloud AI services. The LangGraph workflow successfully coordinated query analysis, planning, validation, generation, citation, and confidence evaluation, with the reflection mechanism proving particularly effective for complex queries requiring additional retrieval iterations. RBAC functioned correctly throughout, and the citation engine produced consistent, traceable references. The system was validated through over 570 automated unit and integration tests covering ingestion, embedding, indexing, retrieval, reranking, citation generation, authentication, streaming, RBAC enforcement, and end-to-end RAG functionality, all of which executed successfully.

Taken together, these outcomes suggest that the largest practical gains over a conventional single-pass RAG chatbot come not from any single component in isolation, but from their combination: hybrid retrieval supplies broader recall across both phrasing styles and exact identifiers, reranking sharpens precision among the resulting candidates, and the reflection loop provides a safety net for the residual cases where even reranked retrieval is insufficient. This layered redundancy is particularly valuable in an institutional setting, where an incorrect or unsupported answer carries greater consequence than in general-purpose conversational use.

TABLE I. SUMMARY OF IMPLEMENTED MODULES

Module	Status
Document Ingestion & Chunking	Completed
Embedding Generation (BGE)	Completed
ChromaDB Vector Store	Completed
Hybrid Retrieval (Dense+BM25+RRF)	Completed
Cross-Encoder Reranking	Completed
RAG Response Generation	Completed
LangGraph Multi-Agent Workflow	Completed
Citation Engine	Completed
Confidence Evaluation	Completed
JWT Auth & RBAC	Completed
FastAPI Backend / Vanilla JS SPA	Completed
Automated Testing (570+ tests)	Completed

TABLE II. TECHNOLOGY STACK

Component	Technology
Language	Python
Backend	FastAPI

Component	Technology
Frontend	HTML5/CSS3/JavaScript (SPA)
Vector DB	ChromaDB
Embedding Model	BAAI/bge-base-en-v1.5
Cross-Encoder	BAAI/bge-reranker-base
LLM	Qwen2.5:7B (Ollama, default) / HF Inference API (optional)
Multi-Agent	LangGraph
Auth	JWT
Testing	Pytest
Deployment	Frontend: Vercel; Backend: Google Colab (T4 GPU)

XI. Novel Contributions

- A comprehensive Agentic AI framework purpose-built for institutional knowledge management.
- Hybrid retrieval integrating dense semantic search with BM25 lexical retrieval.
- Reciprocal Rank Fusion for robust retrieval result aggregation.
- Cross-encoder reranking to improve contextual relevance before generation.
- A citation-aware RAG pipeline grounding every response in official documents.
- A deterministic citation engine supporting traceability and transparency.
- RBAC integrated directly into the retrieval pipeline.
- A committee-based document submission and admin-approval workflow providing human-in-the-loop governance before ingestion.
- Fully local Qwen2.5:7B deployment via Ollama by default, ensuring institutional privacy, with an optional HuggingFace Inference API backend for scalable cloud deployment.
- A LangGraph multi-agent workflow with planning, validation, reflection, and confidence evaluation.
- A modular, scalable architecture supporting future autonomous institutional AI systems.

XII. Future Work

Planned extensions include real-time ERP synchronization, intelligent version management, feedback-driven retrieval optimization, long-term conversational memory, multi-turn institutional reasoning, department-specific agents, voice and multilingual interaction, an administrator analytics dashboard, distributed/multi-node deployment, mobile integration, and autonomous workflow execution via advanced Agentic AI

planning, building on the committee upload and admin-approval portal already implemented.

XIII. Conclusion

This work presented the design, implementation, and evaluation of EduMind, an Agentic AI Assistant for engineering college knowledge management, integrating RAG, hybrid retrieval, semantic embeddings, BM25, RRF, cross-encoder reranking, LangGraph multi-agent reasoning, deterministic citation generation, and RBAC into a unified platform. By grounding responses exclusively in retrieved institutional documents rather than pretrained knowledge, the system significantly reduces hallucination while improving transparency and trust, with deterministic citations enabling direct verification against official sources.

The successful local deployment of embeddings, ChromaDB, hybrid retrieval, Ollama-hosted Qwen2.5:7B, LangGraph orchestration, and secure FastAPI services demonstrates the feasibility of advanced institutional AI without reliance on external cloud services. The modular design supports future expansion toward autonomous agents, multilingual management, enterprise integration, and intelligent administrative automation — providing a scalable, privacy-preserving foundation for next-generation AI-driven knowledge management in higher education.

References

- [1] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *NeurIPS*, vol. 33, 2020.
- [2] T. Chen et al., “Hybrid Search for Dense and Sparse Retrieval,” 2023.
- [3] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Found. Trends Inf. Retr.*, 2009.
- [4] Y. Xiao et al., “BAAI General Embedding Model (BGE),” 2023.
- [5] J. Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *NAACL*, 2019.
- [6] T. Mikolov et al., “Efficient Estimation of Word Representations in Vector Space,” 2013.
- [7] LangChain AI, LangGraph Documentation.
- [8] FastAPI Documentation.
- [9] ChromaDB Documentation.
- [10] Ollama Documentation.
- [11] N. Reimers and I. Gurevych, “Sentence-BERT,” *EMNLP*, 2019.
- [12] C. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge Univ. Press, 2008.
- [13] T. Brown et al., “Language Models are Few-Shot Learners,” *NeurIPS*, 2020.
- [14] IEEE Editorial Style Manual, IEEE Standards Association.